



# From Hours to Minutes

Orchestrating Local LLMs for Sensitive Data Pipelines with  
Apache Airflow



Chhayank Jain · Senior ML Engineer, LLM Systems & Data Pipelines

### WHAT YOU HAVE

- Millions of unstructured documents
- A hard compliance boundary
- A nightly batch, not a chat window

### WHAT YOU NEED

- The same typed rows every night
- An audit trail for every field
- A run that finishes before people arrive

**A three-hour run has already failed. There is no room left to retry it.**

- 1 Data residency you can prove
- 2 A model version you can pin
- 3 Fixed cost, not per token
- 4 No retirement on someone else's calendar

Local is not a purity test. It is a trade.

#### THE BILL

- You own the serving layer
- You own evaluation
- Small models need scaffolding



## PART ONE

# Why the naive pipeline takes hours

## WHERE THE WALL CLOCK GOES



■ 45% GPU idle between requests

■ 25% Loading weights, once per task

■ 12% Re-running after a retry

■ 10% Scheduler overhead

■ 8% Generating tokens

**Eight percent of the run was the model doing work.**

One run, 180 minutes · typical proportions for a naive pipeline, not a single measurement. Profile your own.

## The GPU is idle because nothing is keeping it fed.

### One request at a time

GPU waits between requests



### Continuous batching

Freed slots refill at the next step



time → green = GPU doing work

**23×**

throughput with vLLM over Hugging Face static batching. Continuous batching alone gets ~8×. Paged KV-cache memory management accounts for the rest.



PART TWO

# Seven patterns that got it under two minutes

## Never load model weights inside an Airflow task.

- Workers are short-lived. Weights are not.
- You pay the load cost on every task, every retry, every mapped element.
- One serving process. Every task is a thin HTTP client.

```
# ✗ every task pays the load tax
@task
def extract(doc):
    m = AutoModel.from_pretrained(...)
    return m.generate(doc)

# ✓ the task is a client
@task
def extract(batch):
    return httpx.post(
        LLM_ENDPOINT,          # vLLM / TGI
        json={"prompts": batch},
    ).json()
```

## Dynamic task mapping is a scheduling tool, not a for-loop.

- One task per document makes the scheduler your inner loop.
- Batch first. Then map over the batches.
- Few tasks, each with many requests in flight.

### SIZING THE BATCH

#### Too small

Scheduler overhead dominates.

#### Too large

One failure re-does an hour.

#### 2–10 minutes per mapped task

Batch concurrently inside the task. Throttling requests starves the batcher. Bound the GPU with a Pool.

## The cheapest LLM call is the one you never make.

- Most corpora hold a large share of documents rules can parse.
- A branch task decides. Only the hard ones reach the GPU.

TIER 1

**Rules**

microseconds

TIER 2

**Small model**

classify only

TIER 3

**Full generation**

the expensive path

## Make validation a task, not a line in a log.

- Bind the output to a schema.
- Constrained decoding makes malformed output unsamplable.
- Validate in its own node. Failure becomes a color.

```
class Extraction(BaseModel):  
    patient_id: str  
    admitted_on: date  
    confidence: float  
  
@task.llm(  
    llm_conn_id="local_vllm",  
    output_type=Extraction,  
)  
def extract(doc: str) -> str:  
    return f"Extract from: {doc}"  
  
@task  
def validate(rows): ...
```

## Key the work by what went in, so a retry re-does nothing.

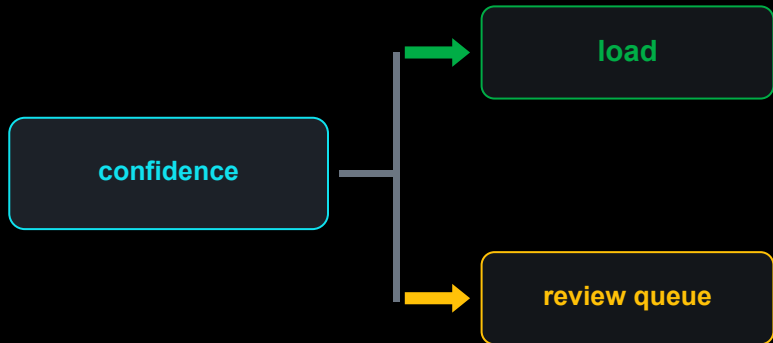
- The default retry regenerates what you already got right.
- Hash document + prompt + model + adapter.
- Retry cost becomes the failures only.

```
def work_key(doc: bytes) -> str:
    return sha256(
        doc
        + PROMPT_VERSION.encode()
        + MODEL_SHA.encode()
        + ADAPTER_SHA.encode()
    ).hexdigest()

@task(retries=3)
def extract(batch):
    todo = [d for d in batch
            if not cache.has(work_key(d))]

    # backfills become cache hits
```

## A model that is unsure is not an error. It is a routing decision.



The run stays green. The uncertain rows stay out of the warehouse.

### WHY THIS BEATS FAILING

#### A red DAG

"Something broke." Nobody can act on that at 3am.

#### A review queue

"340 documents need a person."

#### Alert on the rate

Green DAG, growing backlog is the failure mode. Page above a threshold.

## Version it like data. Schedule on it like data.

- LoRA: as little as 0.01% of the parameters trained.
- Cheap means retrained often.
- A code deploy makes the cheap thing expensive again.

GPT-3 175B at r=4, adapting  $W_q$  and  $W_v$  only: 35MB against a 350GB checkpoint. (Hu et al. 2021)

```
adapter = Asset(
    "s3://models/extract-lora")

@dag(schedule="@weekly")
def finetune():
    @task(outlets=[adapter])
    def train(): ...

# runs when a new adapter lands
@dag(schedule=[adapter])
def extract_documents():
    ...
```

Every pattern in this talk makes another copy. Know where they are.

### XComs

The metadata database. Pass references, not payloads. Or put a custom XCom backend inside the boundary.

### Task logs

A traceback prints the document. Log the hash and the field names, never the content. That includes the model server's own request log.

### The cache

Pattern 5 is a second durable copy of extracted content, with its own retention and erasure obligations.

Airflow's secret masking covers connections and variables, not document content.

1

## Persistent serving

weights load once

2

## A server that batches

the big one

3

## Tiered routing

skip the GPU entirely

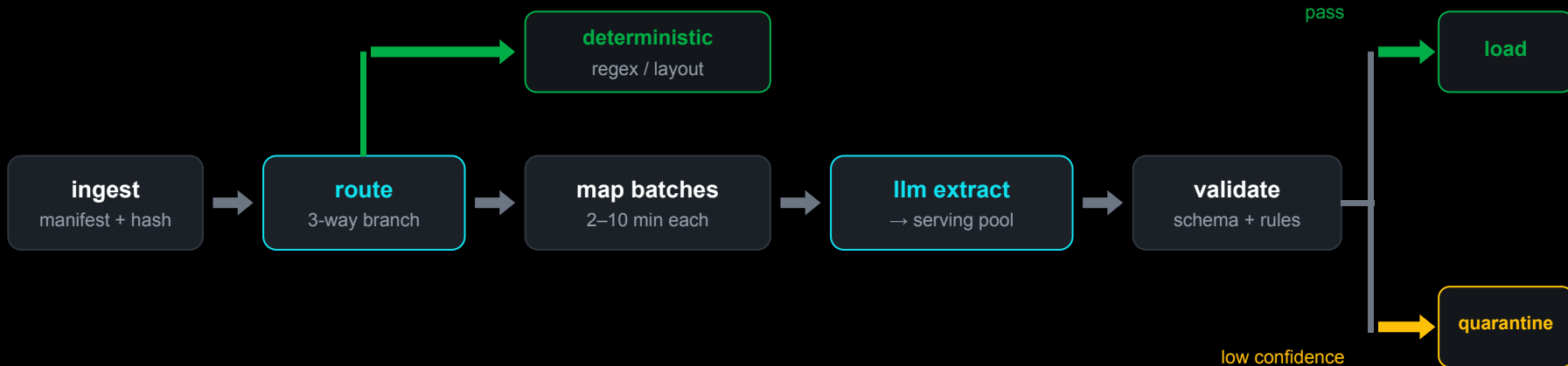
90×

3 hours → under 2 minutes

Measured end to end. Not one of the three is a model change. And parallelism multiplies on top, which is exactly why it is the wrong place to start.

These are not independent factors and they do not cleanly multiply. Persistent serving is a precondition for batching, not a peer to it. This is the order the changes were made in.

## WHAT IT LOOKS LIKE WHEN IT'S DONE



### SERVING POOL

One process, outside the DAG.

### ADAPTER ASSET

Produced by a separate DAG.

Every box is a task. The GPU is not. That is the whole design.

```
from airflow.sdk import dag, task, Asset

adapter = Asset("s3://models/lora")

@dag(schedule=[adapter])
def extract_documents():

    @task.llm(
        llm_conn_id="local_vllm",
        output_type=Extraction,
    )
    def extract(doc: str) -> str:
        return f"Extract from: {doc}"

    extract.expand(doc=batches())
```

### WHAT YOU NO LONGER BUILD

|                                         |           |
|-----------------------------------------|-----------|
| <b>Assets</b>                           | pattern 7 |
| <b>output_type</b>                      | pattern 4 |
| <b>durable=True (@task.agent)</b>       | pattern 5 |
| <b>require_approval (@task.llm)</b>     | pattern 6 |
| <b>enable_hitl_review (@task.agent)</b> | pattern 6 |

**Patterns 1–3 are still yours to design.**

common-ai 0.8.0 at time of recording, still 0.x. Pin it.

 **Retries assume determinism**

Sampling breaks it. You cache, you don't recompute.

 **Evaluation is the hard half**

And it is not an orchestration problem.

 **GPU scheduling is not solved**

Kubernetes will put two pods on a one-model GPU.

 **A prompt edit is a deploy**

It changes every output. Review it like code.

- 1 Airflow orchestrates the model. It does not host it.**
- 2 Measure where the wall clock goes before you optimize.**
- 3 Design for the model being wrong.**



# Questions?



Scan to connect on LinkedIn

**Chhayank Jain · Senior ML Engineer**

[github.com/chhayankjain](https://github.com/chhayankjain)